SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I HARSHITHA S(192325049) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Harshitha S

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

   Answer: Backtracking Search (CSP) Variables:

   D1, D2, D3.

   Domain of each: {Morning (M), Afternoon (A), Night (N)}.

   Variables are tried in the order D1, D2, D3 and values in the order M, A, N.

   Constraints: C1: D1 ≠ N; C2: D2 before D3 (shift(D2) < shift(D3)); C3: all-different (only one doctor per shift); C4: D3 ≠ M; C5: every doctor gets exactly one shift.

   Search trace with constraint checks:

Step	Assignment tried	Constraint check	Result
1	D1 = M	C1 ✓ (not Night); C3 ✓ (Morning free)	Accept → go to D2
2	D1=M, D2 = M	C3 ✗ (Morning already used by D1)	Reject, try next value
3	D1=M, D2 = A	C3 ✓; C2 ✓ possible (D3 can still take Night)	Accept → go to D3
4	D1=M, D2=A, D3 = M	C4 ✗ (D3 cannot work Morning); C3 ✗	Reject, next value
5	D1=M, D2=A, D3 = A	C3 ✗ (Afternoon already used by D2)	Reject, next value
6	D1=M, D2=A, D3 = N	C4 ✓; C3 ✓; C2 ✓ (A < N); C1 ✓; C5 ✓ (all assigned)	SOLUTION FOUND

   Backtracking steps: In step 2, 4 and 5 the current value violated a constraint, so the search backtracked and tried the next value of the same variable. No back-up to D1 was needed, because D1 = M led to a complete consistent assignment.

Remaining search (for completeness): D1=M, D2=N → D3 = M ✗ (C4), D3 = A ✗ (C2: D2 would not be before D3). D1=A, D2=M, D3=N ✓ is a second valid schedule. D1=N ✗ (C1).

Final valid schedule (first solution found):

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

(The other valid schedule is Morning = D2, Afternoon = D1, Night = D3.)

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

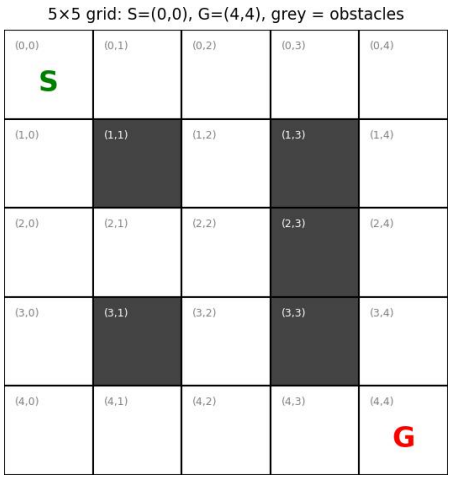
   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Answer:

Assumed grid (the grid figure was not supplied, so a standard 5×5 grid is assumed): cells are written (row, column); Start S = (0,0), Goal G = (4,4); obstacles at (1,1), (1,3), (2,3), (3,1), (3,3).

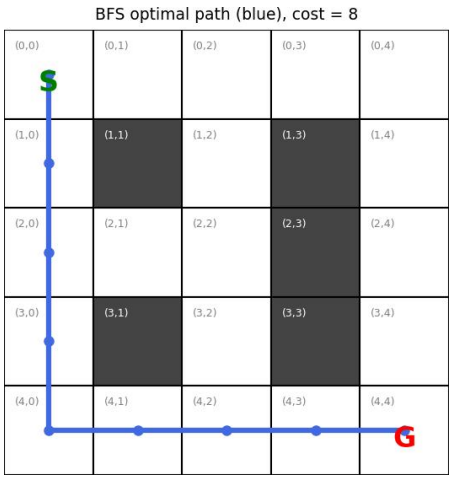


Note on the algorithm: BFS uses a FIFO queue and expands nodes level by level; it does not use a priority or heuristic. The Manhattan distance $h(n) = |r - 4| + |c - 4|$ is therefore shown only for reference (it would be used by Greedy/A*), and it does not change the BFS order. Each move costs 1, so BFS gives the optimal path.

Step-by-step node expansion and queue updates:

Step	Node expanded	h(n)	New nodes added to queue (Up, Down, Left, Right)	Queue after expansion (front → back)
1	(0,0)	8	(1,0), (0,1)	(1,0), (0,1)
2	(1,0)	7	(2,0)	(0,1), (2,0)
3	(0,1)	7	(0,2)	(2,0), (0,2)
4	(2,0)	6	(3,0), (2,1)	(0,2), (3,0), (2,1)
5	(0,2)	6	(1,2), (0,3)	(3,0), (2,1), (1,2), (0,3)
6	(3,0)	5	(4,0)	(2,1), (1,2), (0,3), (4,0)
7	(2,1)	5	(2,2)	(1,2), (0,3), (4,0), (2,2)
8	(1,2)	5	–	(0,3), (4,0), (2,2)
9	(0,3)	5	(0,4)	(4,0), (2,2), (0,4)
10	(4,0)	4	(4,1)	(2,2), (0,4), (4,1)
11	(2,2)	4	(3,2)	(0,4), (4,1), (3,2)
12	(0,4)	4	(1,4)	(4,1), (3,2), (1,4)
13	(4,1)	3	(4,2)	(3,2), (1,4), (4,2)
14	(3,2)	3	–	(1,4), (4,2)
15	(1,4)	3	(2,4)	(4,2), (2,4)
16	(4,2)	2	(4,3)	(2,4), (4,3)
17	(2,4)	2	(3,4)	(4,3), (3,4)
18	(4,3)	1	(4,4)	(3,4), (4,4)
19	(3,4)	1	–	(4,4)
20	(4,4)	0	–	(empty)

Goal (4,4) is reached at step 20. Tracing parents back from G:
Path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)
Moves: Down, Down, Down, Down, Right, Right, Right, Right.



Optimal path cost = 8 moves × 1 = 8. This equals the Manhattan distance from S to G (8), so no shorter path can exist; the path is optimal.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

Answer: Effect of each constraint on decision-making

Constraint	Effect on the robot's decisions
Moves: up, down, left, right	Defines the actions and branching factor (max. 4); no diagonal shortcuts, so paths are Manhattan-type.
Blocked cells	Remove states from the state space; paths must go around them. Unknown blocked cells can make a planned path invalid later, so replanning is needed.
Limited sensing (adjacent cells only)	The environment is partially observable; the robot cannot plan with full knowledge, so it must keep a map (belief state) and sense at every step.
Move cost 1 + risk cost 2	Cost is non-uniform: entering a risky cell costs 3. The shortest path is not always the cheapest, so BFS is not suitable; a cost-based search (UCS) is needed.
Minimize cost with safe navigation	Cost and safety must be balanced; risky cells are avoided unless the detour costs more than the extra 2.
Update knowledge dynamically (online search)	The robot interleaves planning and acting: sense → update map → replan → move.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

Answer : Solution approach – Online Uniform Cost Search with replanning

1. Model the building as a graph: each free cell is a node; step cost = 1, and 1 + 2 = 3 when entering a risky cell.
2. Start with an empty map (the robot knows only S and G). Unknown cells are optimistically assumed free.
3. Sense the four adjacent cells and update the map (blocked / risky).
4. Run Uniform Cost Search from the current cell to G on the known map; UCS expands the lowest g(n) first, so the plan has minimum cost.

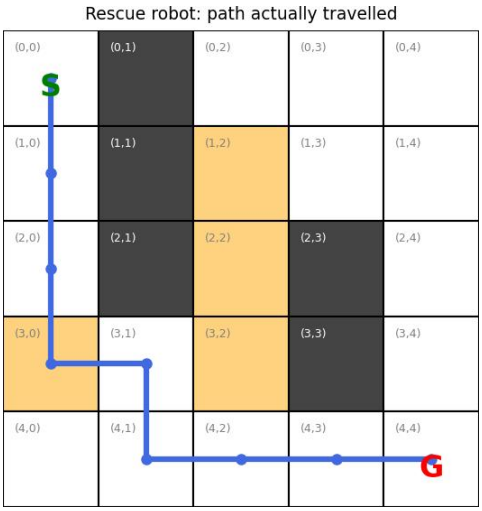
5. Execute only the first move of the plan, then sense again.
6. If a newly found obstacle blocks the plan, discard it and replan from the current cell (back to step 3).
7. Stop when the robot reaches G; the sum of the costs of the moves is the total cost.

Pseudocode:

```
function ONLINE-UCS-RESCUE-AGENT(start, goal) returns total cost
  known_blocked <- {}; known_risky <- {}; pos <- start; total <- 0
  while pos != goal do
      percept <- SENSE(pos)                          // adjacent cells
only
      UPDATE(known_blocked, known_risky, percept)
      plan <- UCS(pos, goal, known_blocked, known_risky)   //
unknown cells assumed free (cost 1)
      if plan = failure then return "no safe path"
      next <- plan[1]                                // execute only
the first step
      total <- total + 1 + (2 if next in known_risky else 0)
      MOVE(next); pos <- next
  return total
```

Worked example (assumed 5×5 building: S = (0,0), G = (4,4); hidden obstacles at (0,1), (1,1), (2,1), (2,3), (3,3); risky cells at (3,0), (3,2), (1,2), (2,2)):

Step	Robot at	Newly sensed	Best plan (UCS) / cost	Move (cost)
1	(0,0)	blocked (0,1)	(0,0) (1,0) (1,1) (1,2) (1,3) (1,4) (2,4) (3,4) (4,4) / 8	(1,0) (1)
2	(1,0)	blocked (1,1)	(1,0) (2,0) (2,1) (2,2) (2,3) (2,4) (3,4) (4,4) / 7	(2,0) (1)
3	(2,0)	risky (3,0), blocked (2,1)	(2,0) (3,0) (3,1) (3,2) (3,3) (3,4) (4,4) / 8	(3,0) (3)
4	(3,0)	nothing new	(3,0) (3,1) (3,2) (3,3) (3,4) (4,4) / 5	(3,1) (1)
5	(3,1)	risky (3,2)	(3,1) (4,1) (4,2) (4,3) (4,4) / 4	(4,1) (1)
6	(4,1)	nothing new	(4,1) (4,2) (4,3) (4,4) / 3	(4,2) (1)
7	(4,2)	nothing new	(4,2) (4,3) (4,4) / 2	(4,3) (1)
8	(4,3)	blocked (3,3)	(4,3) (4,4) / 1	(4,4) (1)



Result: The robot reaches G following (0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4) with total cost = 10 (7 normal moves at cost 1 + 1 move into the risky cell (3,0) at cost 3). The robot replanned each time it discovered a blocked cell. The risky cell (3,0) had to be entered because the only way past the blocked cell (2,1) with the knowledge available was down column 0.

c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

Answer : AI concepts applied

Concept	How it applies to the rescue robot
Intelligent agent	The robot perceives (adjacent cells: free / blocked / risky; its position) through sensors and acts (move up/down/left/right) through actuators. It is a model-based, goal-based and utility-based agent: it keeps a map as internal state, has the goal of reaching the survivor at G, and prefers the least-cost (safest) route. PEAS: Performance = low cost, safe arrival; Environment = building grid; Actuators = wheels; Sensors = adjacent-cell sensors.
Rationality	For every percept sequence the robot selects the action that is expected to maximize its performance measure (minimum total cost, safe). Since it cannot see everything, it acts rationally on the knowledge it has: it picks the cheapest known route, avoids risky cells when a cheaper detour exists and replans when new information appears. Rationality is not omniscience – a new obstacle may force a detour.
Environment type	Partially observable (only adjacent cells), single-agent, deterministic moves (each action has the expected effect), sequential (current moves affect future cost), static (the building does not change while the robot moves – assumed), discrete (grid cells), and initially unknown – so it is an online search problem.

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Answer (d): Justification

- Costs are not equal (1 or 3), so BFS would not guarantee the cheapest path, while UCS always expands the cheapest node first and is optimal and complete for positive costs.
- Because the map is unknown and only adjacent cells are visible, an offline plan is not possible; the online (sense–plan–act) loop lets the robot adapt as soon as new obstacles are discovered.
- Replanning after every step guarantees the robot never follows a path that it already knows is blocked, which keeps the navigation safe.
- The state space is a small grid, so re-running UCS at each step is cheap; heuristic search (A*) could be added to speed up planning, but UCS is simpler and still optimal for the known map.
- DFS and greedy search can be misled into long or risky routes and are not cost-optimal, so they are less suitable.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided